Архитектура аппаратных средств

Путь работы с виртуальной памятью

В данной работе рассмотрен механизм виртуальной памяти: как формируется виртуальный адрес, как он преобразуется в физический, какие структуры данных для этого используются и в какой последовательности происходит обращение к памяти при страничной, сегментной и сегментно-страничной организации.

1) Назначение виртуальной памяти

Оперативная память компьютера ограничена по объёму. При запуске нескольких программ или одной большой программы её может не хватить. Раньше эту проблему решали вручную: программист разбивал программу на части (оверлеи) и сам управлял их загрузкой. Позже появился свопинг - выгрузка целого процесса на диск. Оба подхода неудобны: оверлеи требуют ручной работы, а свопинг перемещает слишком много данных и не позволяет запускать процесс, если его размер больше свободной оперативной памяти.

Виртуальная память решает эти задачи автоматически. Программе кажется, что ей доступен большой непрерывный адресный диапазон (часто в несколько гигабайт), хотя реально в оперативной памяти в каждый момент времени находится только часть этого пространства. Остальное хранится на диске и подгружается по мере необходимости. Адрес, который формирует программа (виртуальный), не совпадает с номером ячейки в оперативной памяти (физическим адресом). Преобразование выполняет аппаратура вместе с операционной системой.

2) Общий путь обращения к памяти

Когда процессор выполняет команду, которая читает или пишет данные, происходит примерно такая последовательность действий:

1) Процессор формирует виртуальный адрес. Он состоит из номера блока (страницы или сегмента) и смещения внутри блока.

2) Аппаратный блок управления памятью (MMU) по таблицам, подготовленным операционной системой, определяет, находится ли нужный блок в оперативной памяти.

3) Если блок есть в оперативной памяти (страница или сегмент «присутствует»), из таблицы берётся его физический базовый адрес, к нему прибавляется смещение - получается физический адрес. По этому адресу выполняется обращение к оперативной памяти (или к кэшу, если данные уже там).

4) Если блока в оперативной памяти нет, возникает прерывание (page fault или segment fault). Управление передаётся операционной системе.

5) Операционная система находит свободную физическую страницу (или освобождает занятую, выгрузив её на диск при необходимости), загружает нужный блок с диска, обновляет таблицу страниц/сегментов и возвращает управление процессу. Процессор повторяет обращение - теперь оно успешно.

Таким образом, путь всегда начинается с виртуального адреса и заканчивается либо успешным доступом к физической ячейке, либо обработкой отказа с последующей повторной попыткой.

3) Страничная организация виртуальной памяти

При страничной организации всё виртуальное адресное пространство процесса делится на блоки одинакового размера - виртуальные страницы. Оперативная память делится на физические страницы (фреймы) того же размера. Размер страницы обычно берётся степенью двойки (4 Кбайт, 8 Кбайт и т.д.), чтобы проще выделять номер страницы и смещение из адреса.

Виртуальный адрес при страничной организации можно представить как пару (p, d), где p - номер виртуальной страницы, d - смещение внутри страницы. Для каждого процесса операционная система

создаёт таблицу страниц. В каждой строке таблицы хранится:

- номер физической страницы (если страница загружена);
- бит присутствия (страница в ОП или на диске);
- бит модификации (менялась ли страница после загрузки);
- бит обращения (обращались ли к странице);
- права доступа (чтение, запись, исполнение);
- при необходимости адрес на диске, если страница выгружена.

Путь преобразования адреса при страничной организации:

1) Из виртуального адреса выделяются номер виртуальной страницы p и смещение d.

2) По адресу таблицы страниц текущего процесса (он хранится в специальном регистре процессора) находится строка с номером p.

3) Проверяется бит присутствия. Если он равен 1, из строки берётся номер физической страницы. Физический адрес = базовый адрес этой страницы + d.

4) Если бит присутствия равен 0, возникает отказ страницы. ОС загружает страницу с диска в свободный фрейм, обновляет таблицу и повторяет доступ.

Таблицы страниц сами могут быть большими, поэтому на практике часто используют многоуровневые таблицы (двухуровневую или более). Тогда преобразование проходит через несколько уровней таблиц, но принцип тот же: по номеру страницы находится физический фрейм, к нему добавляется смещение.

4) Сегментная организация

При сегментной организации программа делится на логические части разного размера - сегменты (код, данные, стек, библиотеки и т.п.). Каждый сегмент может быть размещён в оперативной памяти независимо от других. Виртуальный адрес имеет вид (g, s), где g - номер сегмента, s - смещение внутри сегмента.

Для каждого процесса ОС строит таблицу сегментов (дескрипторы сегментов). В дескрипторе указываются: базовый физический адрес сегмента (если он в памяти), длина сегмента, бит присутствия, права доступа.

Путь преобразования:

1) Из виртуального адреса берутся номер сегмента g и смещение s.

2) По таблице сегментов находится дескриптор сегмента g.

3) Проверяется, что s не превышает длину сегмента (иначе - ошибка).

4) Если сегмент присутствует в памяти, физический адрес = база сегмента + s.

5) Если сегмента нет, возникает отказ. ОС загружает сегмент (или его часть) и обновляет дескриптор.

Недостаток сегментной организации - внешняя фрагментация: после многократной загрузки и выгрузки сегментов разного размера в оперативной памяти остаются «дыры», в которые трудно поместить новый сегмент.

5) Сегментно-страничная организация

Чтобы совместить удобство сегментов и отсутствие фрагментации у страниц, применяют комбинированную схему. Виртуальное пространство делится на сегменты, а каждый сегмент - на страницы фиксированного размера. Оперативная память по-прежнему разбита на физические страницы.

Виртуальный адрес становится трёхкомпонентным: (номер сегмента, номер страницы внутри сегмента, смещение в странице). Для каждого сегмента существует своя таблица страниц. Таблица сегментов хранит адреса этих таблиц страниц.

Путь преобразования адреса:

1) Из адреса выделяются номер сегмента, номер страницы и смещение.

2) По таблице сегментов находится указатель на таблицу страниц данного сегмента.

3) По номеру страницы в таблице страниц находится номер физического фрейма (или фиксируется, что страницы нет в памяти).

4) К базовому адресу фрейма прибавляется смещение - получается физический адрес.

Такая схема чуть медленнее из-за двух обращений к таблицам, но даёт гибкость сегментов и удобство страниц. На практике часто используют кэш преобразований (TLB), чтобы не ходить в таблицы при каждом обращении.

6) Роль операционной системы и аппаратуры

Аппаратура (MMU) выполняет быстрое преобразование адреса по уже заполненным таблицам и генерирует прерывание при отсутствии нужной страницы или сегмента. Операционная система отвечает за всё остальное:

- создаёт и поддерживает таблицы страниц и сегментов для каждого процесса;
- решает, какие страницы держать в оперативной памяти, а какие выгрузить (алгоритмы замещения: LRU, FIFO, часы и др.);
- при отказе страницы выбирает жертву, при необходимости записывает изменённую страницу на диск, загружает нужную страницу и обновляет таблицы;
- управляет файлом подкачки (pagefile, swap-область) на диске;
- обеспечивает защиту: разные процессы не видят чужие страницы, права доступа проверяются при каждом обращении.

Благодаря этому программист работает с «большой» памятью, не задумываясь о реальном объёме ОЗУ и о том, где в данный момент лежат данные - в оперативной памяти или на диске.

7) Краткое сравнение способов

Признак	Страничная	Сегментная	Сегментно-страничная
Размер блоков	фиксированный	переменный	сегменты переменные, страницы фиксированные
Фрагментация	внутренняя (последняя страница)	внешняя	в основном внутренняя
Преобразование адреса	одна таблица страниц	таблица сегментов	таблица сегментов + таблицы страниц
Удобство для программиста	единое пространство	логические части программы	логика сегментов + эффективность страниц

8) Итоговая схема пути работы с виртуальной памятью

Соберём всё в одну цепочку, которую проходит каждое обращение к памяти в системе с виртуальной памятью:

Программа формирует виртуальный адрес → MMU по таблицам (страниц, сегментов или обеих) проверяет присутствие блока → при наличии блока вычисляется физический адрес и выполняется доступ (часто через кэш) → при отсутствии блока генерируется прерывание → операционная система загружает нужный блок с диска, при необходимости вытесняя другой, обновляет таблицы → управление возвращается процессу, обращение повторяется и завершается успешно.

Именно эта цепочка и есть «путь работы с виртуальной памятью». Она скрыта от прикладного программиста, но определяет, как реально используются ресурсы компьютера и почему система может работать с программами, суммарный размер которых превышает объём оперативной памяти.